Battle Tanks
Timing diagrams

Analysis of rendered block diagrams only. Waveforms use the supplied clock/grid/bus/high-low style.

Read this before using the diagrams

Nine independent sheets cover CPU, collision/game response, input conditioning, cooldown, maze/register paths and VGA. Most sheets use 28 visible clock columns. They do not represent one complete game iteration in 28 clocks.

Normal DIV needs 32 iterations. Debounce and cooldown need many more clocks. Long operations and widely separated VGA boundaries use clearly marked // time breaks or independent preconditioned tests. No circuit is sped up.

Some CPU and sync controls conflict across the source diagrams. Explicit timing conventions and limitations are included after the waveform pages. Every signal has a definition. Software-only state is labelled sw.*.

Waveform pages are intentionally wide: zoom for reading; use fit-to-page or poster/tile printing. The PNG and scalable SVG originals are also supplied.

A. RV32IM pipeline — 28 continuous clk_sys cycles — 28 visible columns

B. RV32M — normal division and immediate special-case bypass — 24 visible columns

C. Collision engine and software response — 28 continuous clocks — 28 visible columns

D. Button conditioning, cooldown trigger and expiry — 28 visible columns — 28 visible columns

E. Display pixel priority and blanking — 28 continuous clocks — 28 visible columns

F. VGA horizontal boundaries — visible blank, HS low, HS high, line wrap — 28 visible columns

G. VGA vertical boundaries — vertical blank, VS low, VS high, frame wrap — 28 visible columns

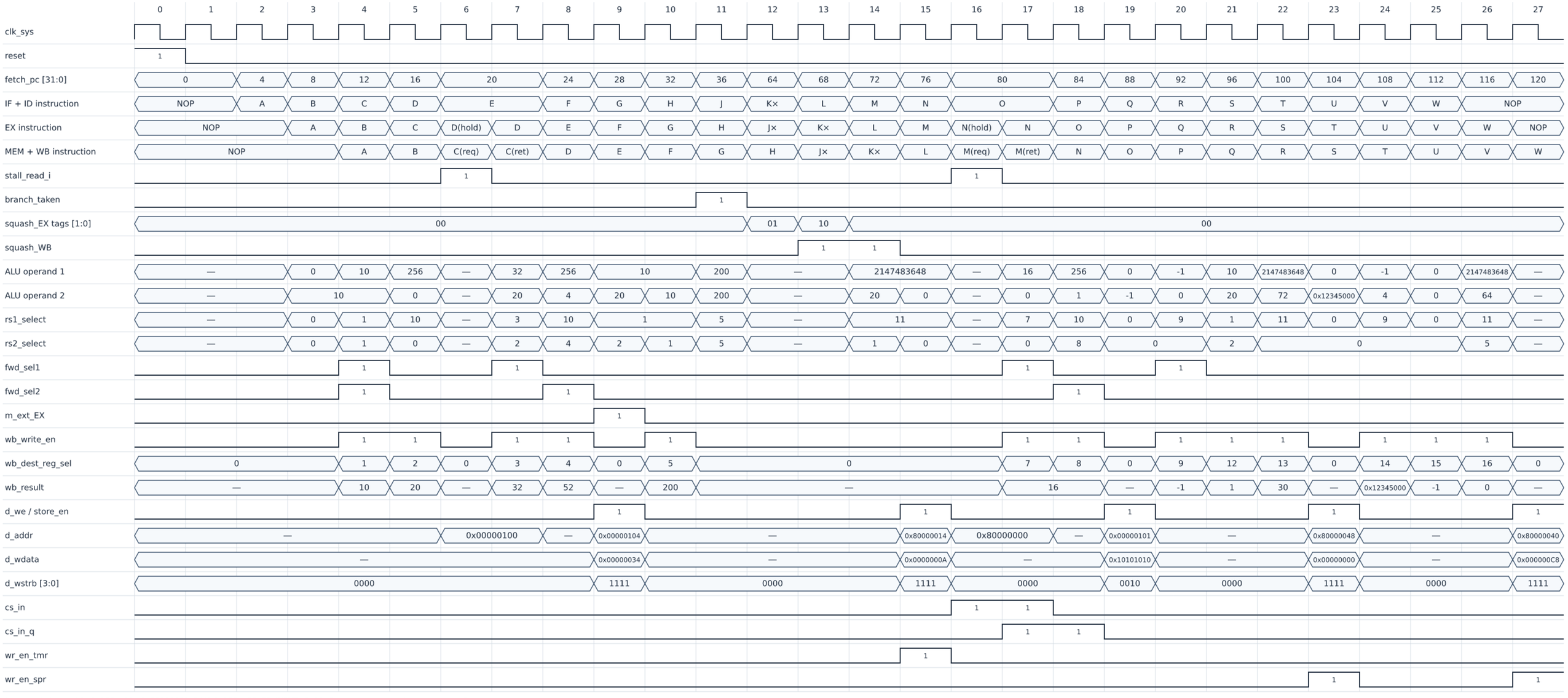
H. Maze coherence, tile boundary, sprite ROM and health HUD — 28 visible columns — 28 visible columns

I. CPU corner cases — loads, halfword lanes, x0, jump and exception detect — 28 visible columns

A. RV32IM pipeline — 28 continuous clk_sys cycles

Reset, forwarding, read wait, taken / not-taken branch, squash, MUL, MMIO and byte store. Selected control assumptions: see README.

Column = after left rising edge; outputs latch at next edge. — = not consumed. // = omitted time / independent boundary window.



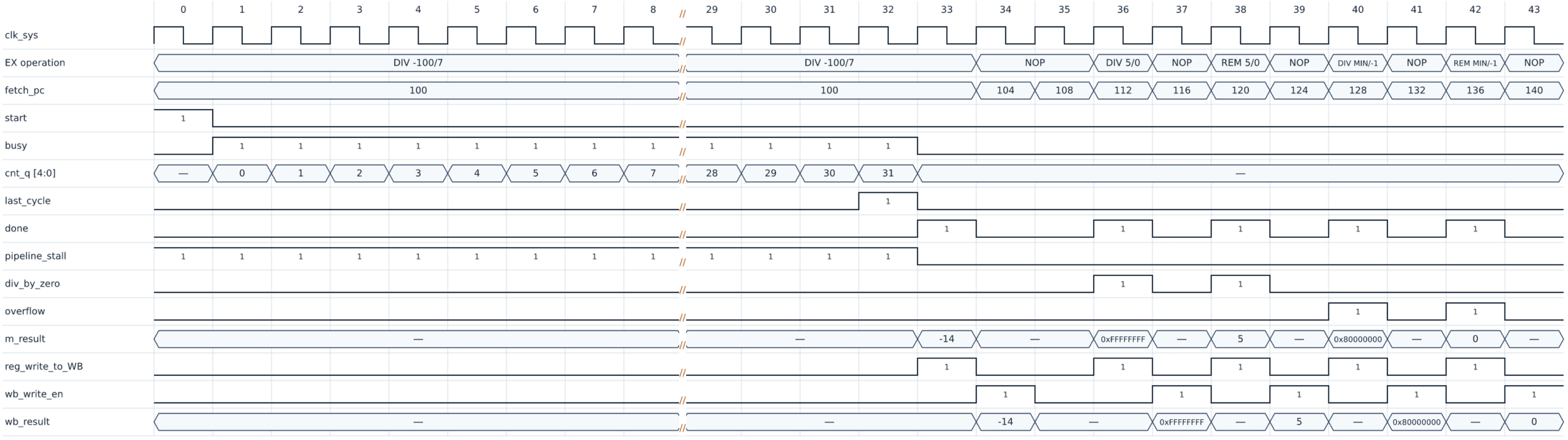
Initial test state: x10=0x100; x11=0x80000000; RAM[0x100]=32; P1_INPUT=0x10. Other consumed initial registers = 0.

H is BEQ at PC 28 -> PC 64. J/K must never commit. Independent illustrative sequence, not the real game software.

B. RV32M — normal division and immediate special-case bypass

24 visible columns; clocks 9–28 OMITTED (20 genuine busy clocks). Normal DIV requires 32 iterations; real elapsed time exceeds 30.

Column = after left rising edge; outputs latch at next edge. — = not consumed. // = omitted time / independent boundary window.



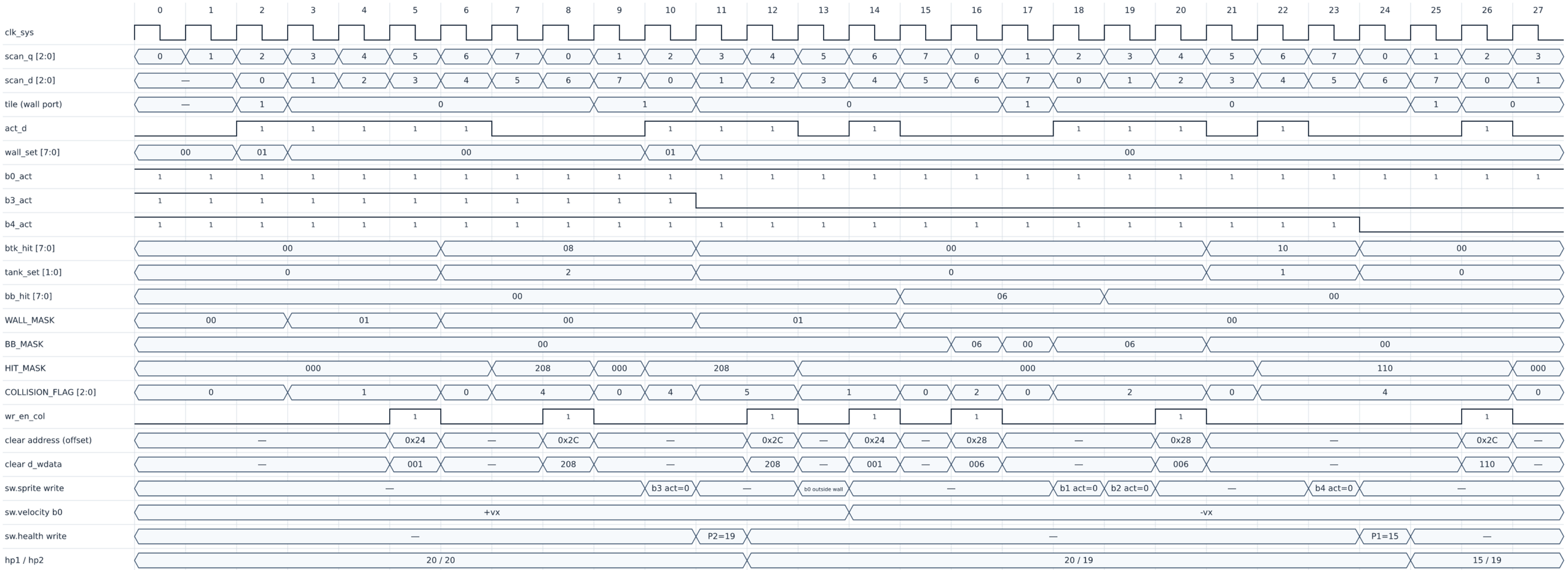
Start column 0; 32 iterations columns 1…32; done 33; WB 34. The // break omits clocks, not work.

DIVU/REMU have same handshake without signed correction. MUL high/low variants use funct3 without divider stall.

C. Collision engine and software response — 28 continuous clocks

Preloaded bullets and maze; arbitrary test position updates. Wall scan, inactive suppression, normal/special tank hit, bullet pair, sticky clear priority.

Column = after left rising edge; outputs latch at next edge. — = not consumed. // = omitted time / independent boundary window.



All masks initially zero; maze and bullets already initialized. No reset is asserted in this independent continuous window.

Scan slot0 at 0 -> wall_set at 2 -> sticky mask at 3. After position write at 13, slot0 scan16 returns floor18.

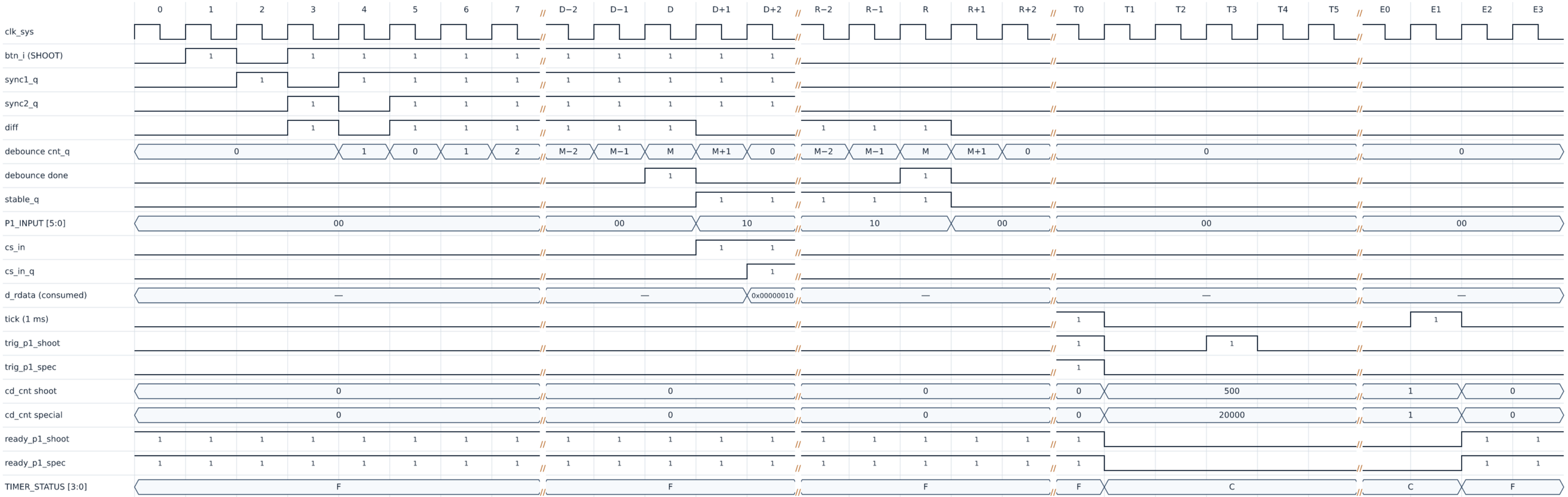
At 8/16 clear wins over a still-live hit; deactivation or separation must precede the final acknowledgement to avoid re-latching.

Travel/reflection inputs are test stimulus. No claim that a bullet traverses a maze, or software performs all these calculations, in 28 clocks.

D. Button conditioning, cooldown trigger and expiry — 28 visible columns

Independent windows separated by //: debounce terminal D; release terminal R; timer trigger T; long cooldown expiry E. 50 MHz assumption.

Column = after left rising edge; outputs latch at next edge. — = not consumed. // = omitted time / independent boundary window.



Debounce gaps hide about 10 ms of sustained mismatch on each transition; M+1 column follows literal current-count comparator timing.

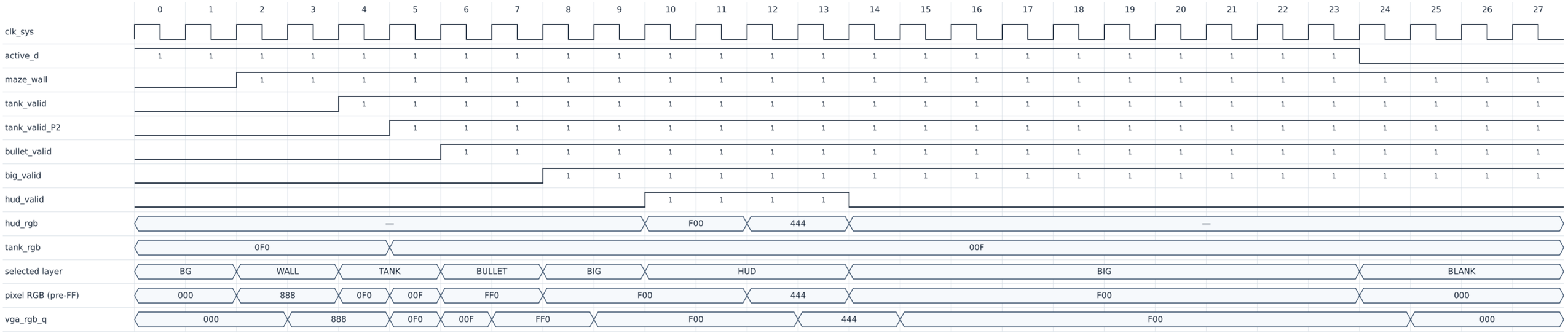
T0 coincides with a real terminal tick: trigger wins over decrement. E is an independent endpoint test with both counters initially 1; not a simultaneous expiry claim.

Normal software must check ready before shooting; hardware reloads even a busy channel if software writes a trigger. No FPGA fire-request pulse is drawn.

E. Display pixel priority and blanking — 28 continuous clocks

Aligned Fig 10 input test: HUD > big bullet > normal bullet > tank > wall > background. Final RGB registered once.

Column = after left rising edge; outputs latch at next edge. — = not consumed. // = omitted time / independent boundary window.



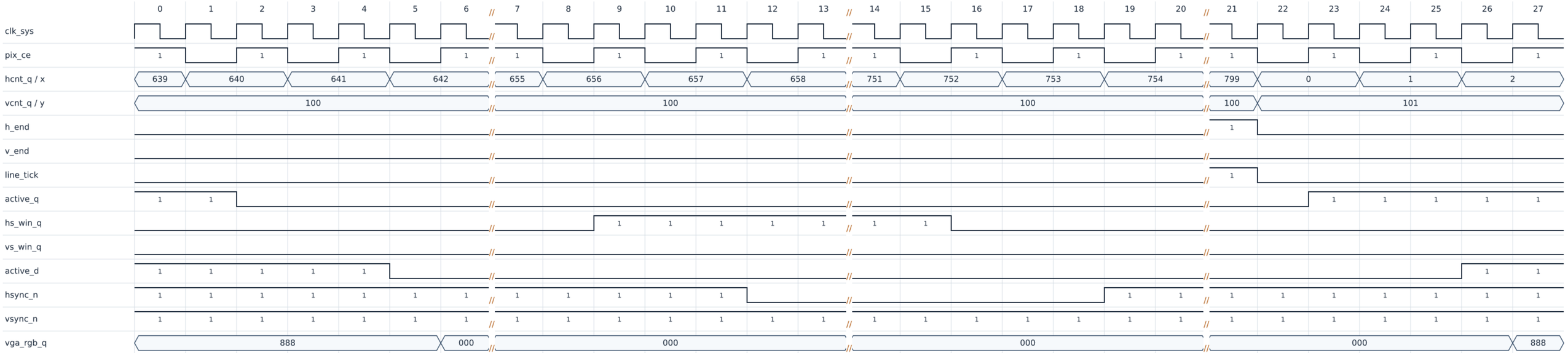
Overlapping valid streams prove priority without claiming a fast full-screen scan. This is a mixer-input test.

Transparent ROM pixels have valid=0 and reveal lower layers. Actual shape ROM bits are not specified in the block diagrams.

F. VGA horizontal boundaries — visible blank, HS low, HS high, line wrap

Four independent seven-clock windows; // separates preconditioned boundary tests. 50 MHz clk_sys, alternating 25 MHz pix_ce.

Column = after left rising edge; outputs latch at next edge. — = not consumed. // = omitted time / independent boundary window.



Sync pin changes five displayed clocks after window start; RGB blank change follows one clock later under literal Fig 10.

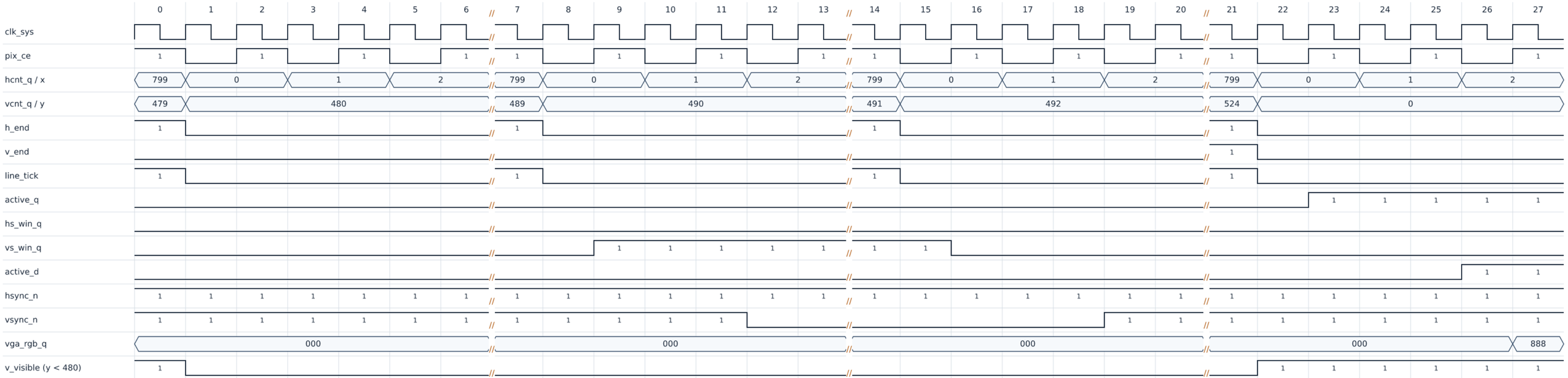
Each window has steady prehistory. // does not skip required counter values within a single continuous trace.

Exact coordinate-to-output alignment is NOT established by these conflicting drawings; verify/reset RTL before relying on pin timing.

G. VGA vertical boundaries — vertical blank, VS low, VS high, frame wrap

Four independent seven-clock windows; // separates preconditioned boundary tests. 50 MHz clk_sys, alternating 25 MHz pix_ce.

Column = after left rising edge; outputs latch at next edge. — = not consumed. // = omitted time / independent boundary window.



Sync pin changes five displayed clocks after window start; RGB blank change follows one clock later under literal Fig 10.

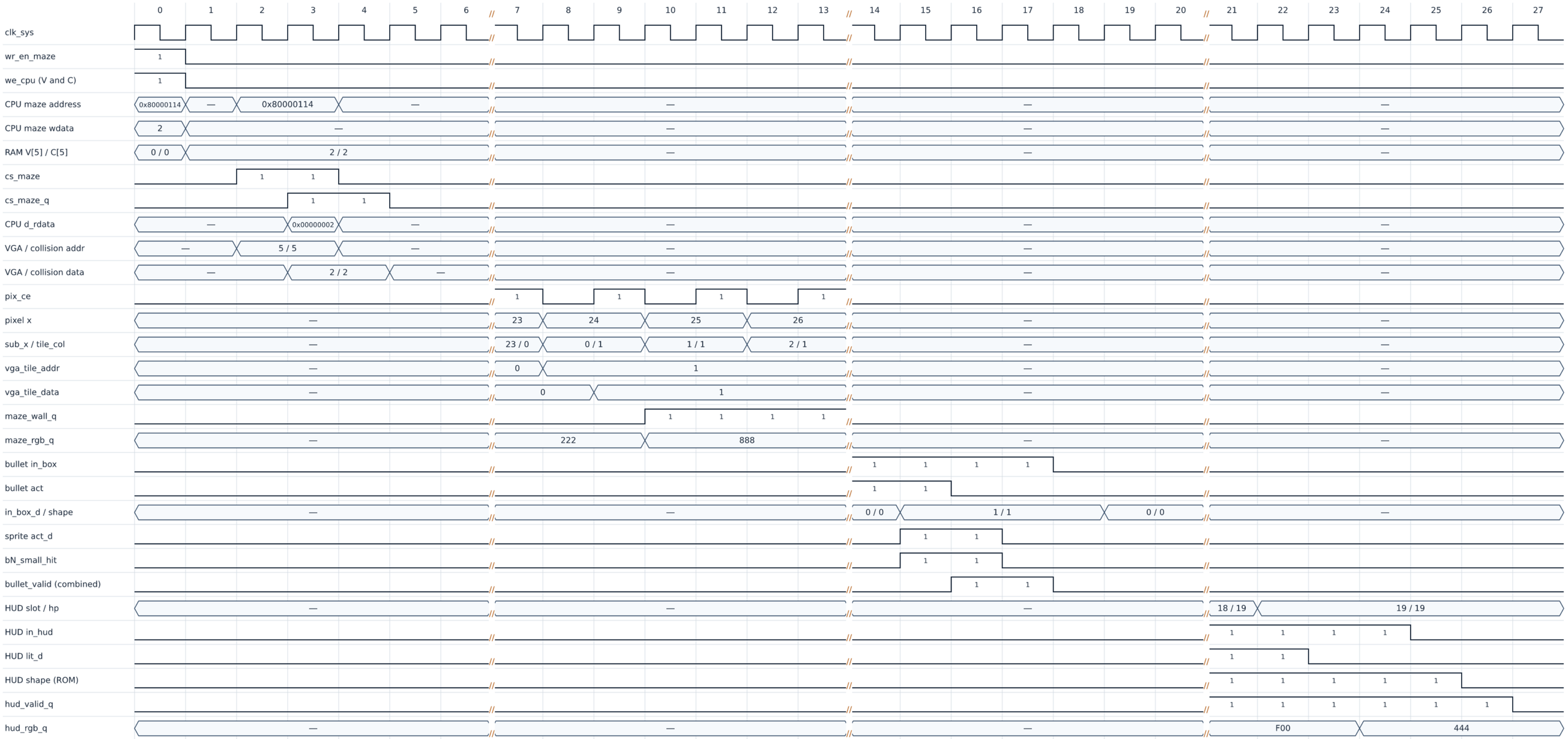
Each window has steady prehistory. // does not skip required counter values within a single continuous trace.

Exact coordinate-to-output alignment is NOT established by these conflicting drawings; verify/reset RTL before relying on pin timing.

H. Maze coherence, tile boundary, sprite ROM and health HUD — 28 visible columns

Independent windows: mirrored maze write/read | 24-pixel tile crossing | bullet visibility | HUD heart valid/colour. No same-address read/write collision tested.

Column = after left rising edge; outputs latch at next edge. — = not consumed. // = omitted time / independent boundary window.



ROM outputs use illustrative shape bits since block diagrams contain no ROM image data. Streams are independent diagnostic tests.

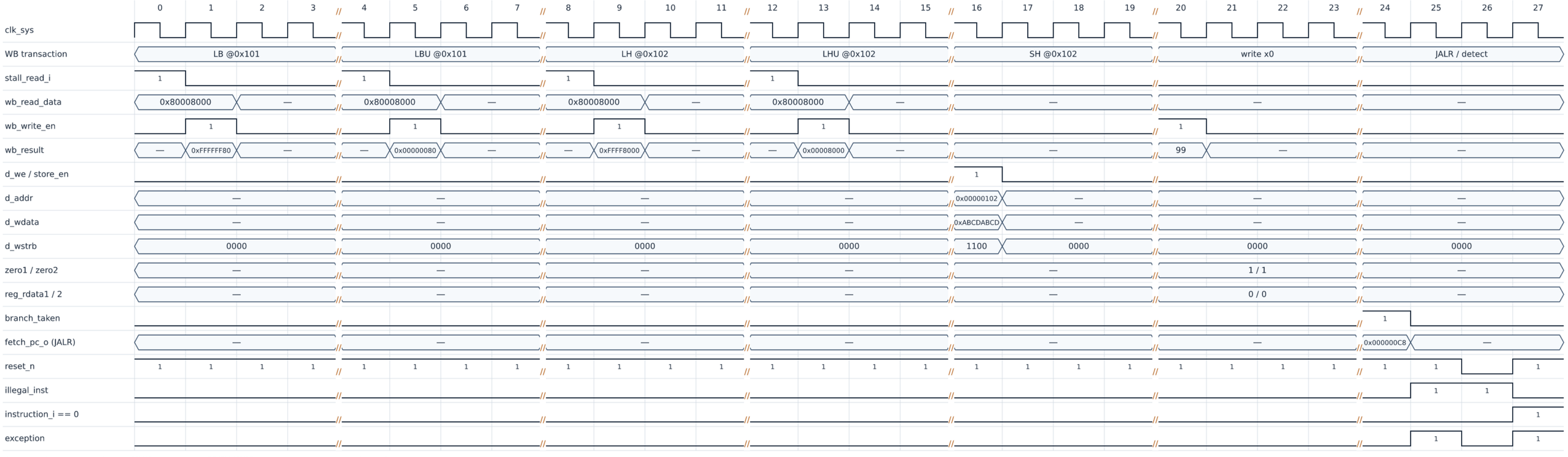
RAM same-address cross-port write/read mode and RAM reset contents are unspecified; this test avoids that collision and assumes preloaded tiles.

Unqualified X tile restart at h_end in Fig11 must be checked when pix_ce=0; boundary tests use a qualified restart convention.

I. CPU corner cases — loads, halfword lanes, x0, jump and exception detect

Seven independent four-clock tests; // separates transactions. Valid inputs established before window. Trap/recovery not specified by diagrams.

Column = after left rising edge; outputs latch at next edge. — = not consumed. // = omitted time / independent boundary window.



Aligned data-memory tests only. MMIO is word-only; SB/SH MMIO behaviour should not be inferred from data-memory lane tests.

Normal ALU add/shift/logic/compare are selected combinationally; numeric examples in sheet A cover ADD, MUL, SLT, OR, LUI, SRAI and XOR.

Battle Tanks — timing diagrams derived from block diagrams only

Scope and reading convention

Only the rendered block diagrams in **../out/** were analysed: Figs 1–12 and P0–P7. No RTL, C program, or diagram-generator code was used as a source of timing behaviour. These are **illustrative test scenarios, not measured simulation waveforms**. Stimulus and assumptions below are necessary because the diagrams do not specify a complete executable program or every control edge.

The sheets follow the supplied photograph: clock at top, numbered cycle columns, boxed bus values, square high/low control waveforms. A column is the interval immediately after its left rising edge. Registered results of activity in column n appear in column n+1. **—** means irrelevant/not consumed, not an electrical unknown; **NOP** is a real bubble/no-side-effect instruction. Numbers are decimal unless prefixed **0x**; masks and RGB values are hexadecimal. Register writes take effect at the right-hand edge of their column.

There is no honest, exhaustive, continuous <30-clock trace of this system. P7b alone requires 32 divider iterations; debounce takes about 500,000 clocks at the pictured 50 MHz assumption; cooldown takes 500 ms or 20 s. Each sheet has fewer than 30 visible columns. Sheets with **//** gaps contain explicitly separated time windows, NOT a faster implementation. The CPU and collision sheets are continuous 28-clock examples. The sheets are independent tests, not successive portions of one 28-clock game run. Cases are representative coverage, not exhaustive verification of every instruction, geometry, or simultaneous event.

Important diagram findings and chosen conventions

1. **FPGA computes and renders the game.** Fig 1 contains an RV32IM CPU and VGA connector, not a host-link/game protocol. A laptop generally cannot accept VGA video directly; a suitable capture interface is needed, or a separately designed FPGA-to-laptop link. None is invented here. The depicted second tank is P2; autonomous enemy AI is not specified.

2. **Reflection and damage are software actions.** Fig 6b scans bullet centres; Fig 6c compares live AABBs; Fig 6d latches flags. No hardware **reflect**, velocity, damage, or game-over signal is drawn. Such annotations in the sheets are labelled **sw.*** or test stimulus.

3. **Collision sticky bits are clear-dominant: q_next = (q | set) & ~clear** (Fig 6d). A persisting collision can set the flag again after a clear. Writing zero cannot clear it. Bullet-bullet sets both involved bits; inactive bullets are excluded. HIT_MASK packs bullet bits [7:0] and tank bits [9:8]. No ownership bit prevents self-hits.

4. **Wall scan takes two clocks to reach wall_set.** The tile-address register and synchronous RAM each add one clock. **scan_d** and **act_d** must match that delay. The sticky bit changes on the following edge. Scanning eight slots does NOT mean a given slot's flag appears immediately or that reflection is automatic. Wall testing is centre-tile sampling, not swept collision or a wall-normal calculation.

5. **MMIO read selects are delayed one clock** (Fig 2b). The sheets assume the low address bits/selected word are held through response consumption; Fig 3/6d show combinational word selection, not a separately registered word address. Stores commit once; read completion does not write a peripheral.

6. **CPU assumptions:** one-cycle synchronous instruction-memory latency, execution-stage register reads with WB forwarding, one wait column for each data read, then a qualified single write-back. IF/ID contains the instruction returned from the preceding fetch PC. On a read wait, PC/IF/ID/ID/EX/EX-WB hold; the held EX instruction has no side effect until release. This is a selected convention, not a uniquely specified implementation.

7. **Branch assumptions:** two younger instructions from the BRAM fetch stream get squash tags. Their register/store/branch effects are suppressed; target instructions are not suppressed. P1's **upd_en**, NOP-mux polarity and P5/P6's EX-versus-WB squash alignment are inconsistent/ambiguous, so literal gate-level branch penalty and combined stall/flush timing cannot be certified. Squash pulses shown are aligned tags under this convention.

8. **Store-path assumption:** effective address = rs1 + immediate, store payload = forwarded rs2, captured separately. P3/P5 label the same ALU operand/result inconsistently for these roles. Byte-lane behaviour follows P6; MMIO is word-only and ignores byte strobes (Fig 2a).

9. **M unit:** select the pictured one-clock multiplier option. Normal DIV/REM uses a start edge, 32 busy iteration edges, a registered done cycle and one WB cycle. P7b special cases bypass iteration: division by zero returns all ones, remainder by zero returns the dividend; signed INT_MIN / -1 returns INT_MIN and remainder zero, without an exception.

10. **Display convention:** use a 50 MHz **clk_sys** and alternating **pix_ce** (25 MHz), consistent with Figs 3/4. Fig 8's frame calculation instead assumes 100 MHz and four clocks/pixel. If your actual system uses 100 MHz, change enable spacing to four. Positive sync-window flags are named **hs_win_q**/**vs_win_q**, delayed three FFs and inverted exactly once at the connector, following Fig 10. Fig 8's polarity annotations are conflicting.

11. **Display latency caveat:** mixer-source tests use already aligned input streams. Fig 10's literal active chain gives **active_d[n] = active_q[n-3]**, and the final RGB register consumes **active_d** one clock later. Sync has only the three shown FFs. This reveals a potential one-clock blank/RGB versus sync alignment discrepancy; the drawings do not establish a fully aligned coordinate-to-connector pipeline. Boundary sheets show that convention, not a claimed fix. Tile restart must be qualified by **pix_ce & h_end**; Fig 11's unqualified X restart can reset early while the pixel counter holds.

Deliverables

- **out/battle_tanks_timing.pdf** — all sheets plus assumptions, instruction key, case explanations and signal definitions.

- **index.html** — browser viewer; **out/*.png** — images; **out/*.svg** — scalable versions.

- **SIGNALS.md** — explanation and source figure for **every plotted signal**.

- **out/timing_values.json** — exact per-column values.

Sheet guide

Sheet · Important cases · Columns

A: CPU pipeline · Reset; add/MUL/logic/shift/compare; both forwarding ports; load waits; taken/not-taken branch; squash blocks wrong-path writes; MMIO; SB · 28 continuous

B: RV32M · Normal DIV stall/start/busy/done/single commit; zero-divisor DIV/REM; signed overflow DIV/REM · 24, with 20 clocks omitted

C: Collisions · Wall/floor; inactive suppression; normal/special tank hit; bullet pair; sticky hold; clear wins set; re-latch; software reflection/removal/health · 28 continuous test clocks

D: Input/timers · Bounce; 2-FF synchronization; accepted press/release; poll return; trigger/tick priority; busy retrigger; hold; expiry/ready · 28, separated windows

E: Pixel mixer · All priorities; P2-over-P1 colour; full/empty hearts; transparent fallback; blank overrides live layers; RGB register · 28 continuous input-test clocks

F: Horizontal VGA · Visible blank; HS low/high; line wrap; enable hold; delayed sync/blank · 28, four windows

G: Vertical VGA · Visible blank; VS low/high; frame wrap; enabled line-end increments · 28, four windows

H: Memory/render · Mirrored maze write/read; tile boundary24; RAM/output delays; bullet ROM/activity delay; heart colour/valid pipeline · 28, four windows

I: CPU corners · LB/LBU/LH/LHU; SH lanes; x0 reads; JALR low-bit clearing; illegal/zero exception detect/reset qualifier · 28, seven tests

Sheet A: illustrative instruction key

Invented to exercise the **pictured hardware**, not recovered from game code. Decimal PC is an instruction address. Initial x10=0x100, x11=0x80000000; RAM[0x100]=32; P1 input=0x10. Other consumed initial registers zero.

Label · PC · Instruction · Purpose / result

A · 0 · **addi x1,x0,10** · x1=10

B · 4 · **add x2,x1,x1** · Both ports forward A; x2=20

C · 8 · **lw x3,0(x10)** · Request6, return/commit7; x3=32

D · 12 · **add x4,x3,x2** · Held at6; load forwarded7; x4=52

E · 16 · **sw x4,4(x10)** · Forward D payload; write52 at WB9

F · 20 · **mul x5,x1,x2** · x5=200; one-clock multiplier

G · 24 · **bne x1,x1,+40** · NOT taken

H · 28 · **beq x5,x5,+36** · Taken EX11; fetch target64 at12

J · 32 · **sw x5,64(x11)** · Wrong-path store MUST NOT occur

K · 36 · **addi x6,x0,99** · Wrong-path register write MUST NOT occur

L · 64 · **sw x1,20(x11)** · TIMER_TRIGGER=0xA, both special channels

M · 68 · **lw x7,0(x11)** · Input request16/return17; x7=16

N · 72 · **addi x8,x7,0** · Held16; input forwarded17; x8=16

O · 76 · **sb x8,1(x10)** · Byte0x10 into lane1; strobe0010

P · 80 · **addi x9,x0,-1** · x9=0xFFFFFFFF

Q · 84 · **slt x12,x9,x0** · Signed -1<0 gives1

R · 88 · **or x13,x1,x2** · 10 OR20 =30

S · 92 · **sw x0,72(x11)** · Bullet0 inactive

T · 96 · **lui x14,0x12345** · x14=0x12345000

U · 100 · **srai x15,x9,4** · -1 remains -1

V · 104 · **xor x16,x0,x0** · x16=0

W · 108 · **sw x5,64(x11)** · Correct-path P1 position value200

IF/ID follows fetch PC by one column. Taken EX11 still leaves K (BRAM return of PC36) in IF/ID12. J/K effects are suppressed; target L executes14. Load completion qualification is assumed: a held raw write-enable must not commit invalid data repeatedly.

Sheet C: important transitions

- **Wall:** scan slot0 at0 □ wall_set2 □ sticky WALL_MASK3. Clear5 removes it at6; later scan re-detects10. Software flips its stored velocity and writes an outside-wall position13; scan16 returns floor18. Final clear14 stays clear. Hardware detects a non-floor centre tile, **not wall orientation**. This test assumes a vertical wall. Horizontal reflection would flip vy; corner/oblique response needs a software policy not shown.

- **Normal hit:** bullet3 overlaps P2 during6–10 □ HIT_MASK=0x208 at7. Clear8 wins one edge, then overlap re-latches10. Software removes bullet10, writes health19 at11 (visible12), clears12. Separation before final acknowledgement avoids re-latching and repeated damage.

- **Bullet pair:** bullets1/2 overlap15–18 □ BB_MASK=0x06 at16. Clear16 wins; re-latches18. Removed by writes18/19; clear20 persists.

- **Special hit:** big bullet4 overlaps P1 during21–23 □ HIT_MASK=0x110 at22. Removed23, P1 health15 written24 (visible25), cleared26. Five-point damage is a CPU-requested health write, not collision arithmetic.

- **Self-hit:** no ownership field; a reflected live bullet overlapping its own tank has the same hit timing. Fig6c treats tanks as active even at software health zero.

Limits: cases that cannot be reconstructed

The diagrams do not specify enemy AI, bullet velocity registers, actual instruction order, aim updates, full-slot policy, bullet lifetime counters, game-over/respawn FSM, health clamp, wall-normal/corner reflection, or exception-handler timing. Health=0 uses the same Fig5 register write as other health values, but no hardware **game_over** signal exists in these drawings. No-ready firing/full slots/lifetime are software decisions not to write an active bullet (or to write act=0). Hardware reloads a cooldown on any trigger regardless of ready.

AABB uses strict overlap: mere edge touching is not a hit; act=0 suppresses hits. No swept collision prevents tunnelling; centre-wall sampling may miss a bullet edge touching a neighbouring tile. Several collision types can set independent masks simultaneously. Software ordering/duplicate damage is unspecified. HIT_MASK aggregates bullets and tanks, not each bullet□tank pairing, so multiple hits can need extra bookkeeping.

RAM reset contents and same-address cross-port read/write modes are unspecified. ROM bits cannot be inferred. Reset/stall priority, flush-chain enable, stalled branches, and exception recovery need RTL confirmation. W1C semantics are clear for the event masks used here; aggregate0x20 clear mapping is not fully drawn. These are representative cases, not exhaustive program verification.

Reproduce and check

python3 make_timing.py generates SVG/JSON/signal definitions/HTML with the Python standard library. **python3 test_timing.py** checks cycle lengths, CPU sequencing, write qualification, real divider count, scan delay, sticky priority, debounce, timers, VGA recurrences and rendering pipelines. These check **our diagram data**, not RTL.

PNG: **node export_png.cjs /path/to/@resvg/resvg-js /path/to/font.ttf**. PDF (Pillow + ReportLab required): **python3 export_pdf.py**. Prebuilt outputs are included, so no installation is needed to view them.

Timing-sheet signal dictionary

Every sheet shares **clk_sys**: the FPGA system clock. High/low square wave; left edge of each column is rising. All timing labels are illustrative, not RTL simulation.

A. RV32IM pipeline — 28 continuous clk_sys cycles

Reset, forwarding, read wait, taken / not-taken branch, squash, MUL, MMIO and byte store. Selected control assumptions: see README.

Signal · Meaning / source

reset · Active-high internal CPU reset. Establishes PC=0 and bubbles; reset_n = NOT reset. P1/P2.

fetch_pc [31:0] · Instruction BRAM address, not IF/ID instruction PC. Holds on read wait; redirects to 64 after H. P1.

IF + ID instruction · Returned instruction decoded now; fetch PC is one column ahead. × marks wrong-path work.

EX instruction · Executing instruction, or held behind a read wait. P2/P3/P7.

MEM + WB instruction · req/ret are request/completion of the SAME load, not two instructions. P5/P6.

stall_read_i · Chosen one-column data-read wait; pipeline holds. Load commits only on return. P1/P5/P6.

branch_taken · H taken in EX; G at 10 NOT taken. Wrong-path redirects suppressed. P5.

squash_EX tags [1:0] · Semantic younger-instruction tags corresponding to P1 first/second squash. Raw alignment ambiguous.

squash_WB · Semantic squash qualifier disables J/K WB effects. Annotation, not an additional drawn FF.

ALU operand 1 · Consumed EX operand 1. — for held/squashed work. P3/P4.

ALU operand 2 · EX register/immediate; store offset is NOT payload. Separate store payload assumed. P3.

rs1_select · EX operand source register. x0 forced to zero. P4.

rs2_select · Second source or store-payload register; immediate instructions have no consumed rs2. P4.

fwd_sel1 · WB-to-EX forwarding: A□B, C□D, M□N, P□Q. P4; EX reads assumed.

fwd_sel2 · WB-to-EX source 2/store forwarding: A□B, D□E, N□O. P4.

m_ext_EX · F is MUL. ALU mux input 4; chosen one-clock multiplier, no DIV stall. P7a/P7c.

wb_write_en · Qualified architectural register write; zero for load request/stores/branches/squash. P4/P6.

wb_dest_reg_sel · Destination when write enabled; zero here means no write.

wb_result · Consumed ALU/MUL/load write-back value. P6.

d_we / store_en · Actual WB write pulse. Squashed J cannot write; read wait cannot repeat a store. P6/Fig 2a.

d_addr · Data/MMIO byte address held across load response; store effective address distinct from payload. P5/Fig 2a.

d_wdata · Lane-replicated payload; SB drives 0x10101010 but writes only enabled byte. P6.

d_wstrb [3:0] · Data-memory byte enables; MMIO ignores strobes and requires word access. P6/Fig 2a.

cs_in · Input peripheral address selected; selection does not imply a write. Fig 2a.

cs_in_q · One-clock registered input select; only column 17 response is consumed. Fig 2b.

wr_en_tmr · Timer selection AND d_we; L triggers both special channels with 0xA. Figs 2a/4.

wr_en_spr · Sprite selection AND d_we; S clears bullet 0, W updates P1 position. Figs 2a/5.

- Initial test state: x10=0x100; x11=0x80000000; RAM[0x100]=32; P1_INPUT=0x10. Other consumed initial registers = 0.

- H is BEQ at PC 28 □ PC 64. J/K must never commit. Independent illustrative sequence, not the real game software.

B. RV32M — normal division and immediate special-case bypass

24 visible columns; clocks 9–28 OMITTED (20 genuine busy clocks). Normal DIV requires 32 iterations; real elapsed time exceeds 30.

Signal · Meaning / source

EX operation · MIN = signed 0x80000000. Special cases bypass iteration. P7b.

fetch_pc · PC held until done then increments. P7c hold convention.

start · Pulse loads abs(dividend)=100 and clears remainder before BUSY. P7b.

busy · High during 32 real iteration columns 1–32; special cases never BUSY. P7b.

cnt_q [4:0] · Iteration index 0–31. Omitted clocks contain indices 8–27. P7b.

last_cycle · cnt_q=31 during column 32; final iteration completes on next edge. P7b.

done · Normal registered completion at 33; immediate bypass for zero/overflow. P7b.

pipeline_stall · is_div_op AND NOT done. High at start and BUSY; freezes PC/IF/ID/EX. P7b/P7c.

div_by_zero · DIV gives all ones; REM gives dividend. No trap. P7b.

overflow · Signed MIN/-1: DIV gives MIN, REM gives zero. Not an exception. P7b.

m_result · Selected result; normal signed quotient truncates toward zero. P7b/P7c.

reg_write_to_WB · Enabled only when done. Busy columns insert no-write bubbles. P7c.

wb_write_en · Single commit one clock after result capture, no repeated busy write. P4/P6.

wb_result · Committed results. Normal REM -100/7 would use same stall and return -2.

- Start column 0; 32 iterations columns 1…32; done 33; WB 34. The // break omits clocks, not work.

- DIVU/REMU have same handshake without signed correction. MUL high/low variants use funct3 without divider stall.

C. Collision engine and software response — 28 continuous clocks

Preloaded bullets and maze; arbitrary test position updates. Wall scan, inactive suppression, normal/special tank hit, bullet pair, sticky clear priority.

Signal · Meaning / source

scan_q [2:0] · Free-running selected bullet slot 0–7. Fig 6b.

scan_d [2:0] · Slot identity delayed two clocks to match address register plus maze BRAM. Fig 6b.

tile (wall port) · Two-clock centre-tile result; 0=floor, 1=wall. Slot7 is in a wall but inactive. Fig 6b/7.

act_d · Sampled activity delayed two clocks. Inactive wall slot7 cannot set a hit. Fig 6b.

wall_set [7:0] · One-hot mask when delayed slot is active and tile nonzero. Slots0 set at 2 and 10. Fig 6b.

b0_act · Bullet0 remains alive through reflection; hardware does not remove it on wall hit. Fig 5.

b3_act · Normal bullet3 removed by a software register write at 10, effective 11. Fig 5.

b4_act · Special bullet4 removed by write at 23, effective 24. Fig 5.

btk_hit [7:0] · Live AABB hits: bullet3□P2, later bullet4□P1. Normal=6×6; special=12×12. Figs 6a/6c.

tank_set [1:0] · P1 bit0/P2 bit1. Both tanks always collision-active; no ownership filter. Fig 6c.

bb_hit [7:0] · Overlap of active bullets1/2 sets BOTH bits (0x06). Deactivating either stops the live overlap. Fig 6c.

WALL_MASK · Sticky per-bullet wall mask; persists after detection. Cleared at 5 and 14. Fig 6d.

BB_MASK · Sticky bullet-pair mask. Simultaneous clear/set at 16 clears; persistent overlap sets again at 18. Fig 6d.

HIT_MASK · Sticky {tank[1:0],bullet[7:0]}. Clear at 8 wins; still-live hit re-sets at 10. Later 0x110 identifies special bullet4/P1. Fig 6d.

COLLISION_FLAG [2:0] · bit0 any wall; bit1 any BB; bit2 any bullet-tank. ORs of sticky masks, NOT single-clock hit pulses. Fig 6d.

wr_en_col · Selected collision W1C MMIO write. Clear priority shown at 8 and 16. Fig 6d.

clear address (offset) · MMIO low offset; full base is 0x80000000. Fig 2a/6d.

clear d_wdata · Written one bits clear corresponding sticky bits; zeros do nothing. Fig 6d.

sw.sprite write · Illustrative software commits, not collision outputs. Precomputed CPU actions; actual C latency unspecified. Fig 1/5.

sw.velocity b0 · SOFTWARE STATE, not a drawn FPGA port. Example vertical-wall reflection flips vx at the position-update edge.

sw.health write · Illustrative CPU HEALTH_BAR_REG writes: normal damage 1, special damage 5. Hardware only stores the requested word. Fig 1/5.

hp1 / hp2 · Registered health values after CPU writes; no hardware health decrement occurs. Fig 5/12.

- All masks initially zero; maze and bullets already initialized. No reset is asserted in this independent continuous window.

- Scan slot0 at 0 □ wall_set at 2 □ sticky mask at 3. After position write at 13, slot0 scan16 returns floor18.

- At 8/16 clear wins over a still-live hit; deactivation or separation must precede the final acknowledgement to avoid re-latching.

- Travel/reflection inputs are test stimulus. No claim that a bullet traverses a maze, or software performs all these calculations, in 28 clocks.

D. Button conditioning, cooldown trigger and expiry — 28 visible columns

Independent windows separated by //: debounce terminal D; release terminal R; timer trigger T; long cooldown expiry E. 50 MHz assumption.

Signal · Meaning / source

btn_i (SHOOT) · Async physical button. Initial bounce is rejected. Button sampled between edges; no metastability simulated. Fig 3.

sync1_q · First synchronizer flip-flop, one edge after sampled raw input. Fig 3.

sync2_q · Second synchronizer flip-flop, one further edge later. Fig 3.

diff · sync2_q XOR stable_q; counter resets whenever input returns to accepted level. Fig 3.

debounce cnt_q · M=DEBOUNCE_MAX=500000 at 50 MHz. Terminal comparator reads current count; stable changes on following edge. Fig 3.

debounce done · Current count equals M. One pulse at press/release endpoint; not a shoot-command pulse. Fig 3.

stable_q · Accepted level, not edge event. Press accepted D+1; release R+1. Software detects an edge by comparing polls. Fig 3.

P1_INPUT [5:0] · SHOOT is packed bit4; all other stimulus buttons zero. P2 input held zero. Fig 3.

cs_in · Read selection held across response while polling the accepted press. Fig 2a.

cs_in_q · One-clock registered read select in the shown input window; last held select continues into omitted time. Fig 2b.

d_rdata (consumed) · Input load response consumed at D+2; low word address held through read. Figs 2b/3.

tick (1 ms) · Terminal prescaler pulse, real ticks separated by 50000 clocks. T0 exercises simultaneous tick/trigger; E is an independent terminal-counter test. Fig 4.

trig_p1_shoot · T0 triggers shoot and special; T3 deliberately retriggers shoot while not ready. Hardware does not gate trigger by ready. Fig 4.

trig_p1_spec · T0 starts 20-second special cooldown. Trigger takes priority over tick/decrement. Fig 4.

cd_cnt shoot · 15-bit channel reload = 500 ms; holds without tick; nonzero counter decrements on tick. Fig 4.

cd_cnt special · 15-bit special channel reload = 20000 ms. E window jumps to final remaining millisecond. Fig 4.

ready_p1_shoot · High exactly when shoot counter zero. Counter does not underflow. Fig 4.

ready_p1_spec · High exactly when special counter zero. P2 channels idle/ready throughout. Fig 4.

TIMER_STATUS [3:0] · bit0 P1 shoot, bit1 P1 special, bit2 P2 shoot, bit3 P2 special. Fig 4.

- Debounce gaps hide about 10 ms of sustained mismatch on each transition; M+1 column follows literal current-count comparator timing.

- T0 coincides with a real terminal tick: trigger wins over decrement. E is an independent endpoint test with both counters initially 1; not a simultaneous expiry claim.

- Normal software must check ready before shooting; hardware reloads even a busy channel if software writes a trigger. No FPGA fire-request pulse is drawn.

E. Display pixel priority and blanking — 28 continuous clocks

Aligned Fig 10 input test: HUD > big bullet > normal bullet > tank > wall > background. Final RGB registered once.

Signal · Meaning / source

active_d · Delayed active qualifier at mixer input. Low blanks RGB regardless of layer valids. Fig 10.

maze_wall · Aligned wall valid. Floor RGB computed by Fig 11 does not override BG in the pictured mixer. Figs 10/11.

tank_valid · Combined tank valid after sprite ROM/combining register. Fig 9; input alignment assumed.

tank_valid_P2 · Both tanks hit the test pixel: P2 wins colour selection. Supplied as aligned test stimulus. Fig 9.

bullet_valid · Normal bullet pixel valid overrides tank/wall. Figs 9/10.

big_valid · Special bullet pixel valid overrides normal bullet. Figs 9/10.

hud_valid · Heart shape pixel has highest priority. Full and empty hearts both valid; health changes colour only. Figs 10/12.

hud_rgb · Full red then empty grey heart test pixels; prealigned colour. Fig 12.

tank_rgb · P1 green then P2 blue on overlapping masks. Fig 9.

selected layer · Diagnostic annotation for combinational priority; not an extra hardware bus. Fig 10.

pixel RGB (pre-FF) · 12-bit value after blank mux, hex RGB444. Fig 10.

vga_rgb_q · One-clock registered output. active_d low at 24 □ RGB black at 25. Fig 10.

- Overlapping valid streams prove priority without claiming a fast full-screen scan. This is a mixer-input test.

- Transparent ROM pixels have valid=0 and reveal lower layers. Actual shape ROM bits are not specified in the block diagrams.

F. VGA horizontal boundaries — visible blank, HS low, HS high, line wrap

Four independent seven-clock windows; // separates preconditioned boundary tests. 50 MHz clk_sys, alternating 25 MHz pix_ce.

Signal · Meaning / source

pix_ce · Counter enable, not a clock. Only enabled edges advance counters. Fig 8.

hcnt_q / x · Horizontal counter 0–799; held on disabled pixel clocks. Fig 8.

vcnt_q / y · Vertical counter advances on line_tick only; 524□0 at frame wrap. Fig 8.

h_end · Current horizontal count equals 799; may span two clocks. Fig 8.

v_end · Current vertical count equals 524; requires line_tick for frame wrap. Fig 8.

line_tick · pix_ce AND h_end; advances vertical counter once per line. Fig 8.

active_q · Registered visible test h<640 AND v<480. Fig 8.

hs_win_q · Positive registered 656<=h<752 window; one chosen consistent polarity. Figs 8/10.

vs_win_q · Positive registered 490<=v<492 window. Figs 8/10.

active_d · Literal three-FF delay of active_q before blank mux. Fig 10.

hsync_n · NOT hs_win_q delayed three clocks, exactly one inversion. Fig 10.

vsync_n · NOT vs_win_q delayed three clocks, exactly one inversion. Fig 10.

vga_rgb_q · RGB register consumes active_d one further clock later; steady nonblank test colour 888. Fig 10.

- Sync pin changes five displayed clocks after window start; RGB blank change follows one clock later under literal Fig 10.

- Each window has steady prehistory. // does not skip required counter values within a single continuous trace.

- Exact coordinate-to-output alignment is NOT established by these conflicting drawings; verify/reset RTL before relying on pin timing.

G. VGA vertical boundaries — vertical blank, VS low, VS high, frame wrap

Four independent seven-clock windows; // separates preconditioned boundary tests. 50 MHz clk_sys, alternating 25 MHz pix_ce.

Signal · Meaning / source

pix_ce · Counter enable, not a clock. Only enabled edges advance counters. Fig 8.

hcnt_q / x · Horizontal counter 0–799; held on disabled pixel clocks. Fig 8.

vcnt_q / y · Vertical counter advances on line_tick only; 524□0 at frame wrap. Fig 8.

h_end · Current horizontal count equals 799; may span two clocks. Fig 8.

v_end · Current vertical count equals 524; requires line_tick for frame wrap. Fig 8.

line_tick · pix_ce AND h_end; advances vertical counter once per line. Fig 8.

active_q · Registered visible test h<640 AND v<480. Fig 8.

hs_win_q · Positive registered 656<=h<752 window; one chosen consistent polarity. Figs 8/10.

vs_win_q · Positive registered 490<=v<492 window. Figs 8/10.

active_d · Literal three-FF delay of active_q before blank mux. Fig 10.

hsync_n · NOT hs_win_q delayed three clocks, exactly one inversion. Fig 10.

vsync_n · NOT vs_win_q delayed three clocks, exactly one inversion. Fig 10.

vga_rgb_q · RGB register consumes active_d one further clock later; steady nonblank test colour 888. Fig 10.

v_visible (y < 480) · Diagnostic vertical-visible compare. At line479□480 it falls; active_q is already low because h=799 is horizontally blank. Fig 8.

- Sync pin changes five displayed clocks after window start; RGB blank change follows one clock later under literal Fig 10.

- Each window has steady prehistory. // does not skip required counter values within a single continuous trace.

- Exact coordinate-to-output alignment is NOT established by these conflicting drawings; verify/reset RTL before relying on pin timing.

H. Maze coherence, tile boundary, sprite ROM and health HUD — 28 visible columns

Independent windows: mirrored maze write/read | 24-pixel tile crossing | bullet visibility | HUD heart valid/colour. No same-address read/write collision tested.

Signal · Meaning / source

wr_en_maze · CPU maze write decode. Chosen tile5 address is in range. Fig 2a/7.

we_cpu (V and C) · Single range-qualified pulse drives BOTH maze copies. CPU is not stalled by independent read ports. Fig 7.

CPU maze address · Tile5 address = 0x80000100+4×5. Held for read at 2/3. Fig 7.

CPU maze wdata · Low two bits written: 2=breakable wall. Both copies update at right edge of column0. Fig 7.

RAM V[5] / C[5] · Mirrored stored tile contents; initialized floor then coherent update to breakable. Diagnostic memory-state annotation. Fig 7.

cs_maze · CPU read selects maze; only write pulse at0 alters copies. Fig 2a.

cs_maze_q · One-clock registered select; response consumed at3. Fig 2b.

CPU d_rdata · One-clock CPU BRAM read, zero-extended tile. Low address held through response. Fig 7/2b.

VGA / collision addr · Independent read ports both request tile5 after write completed; no arbitration. Fig 7.

VGA / collision data · Synchronous read data one clock after port addresses. Concurrent same-address write/read behaviour is NOT specified. Fig 7.

pix_ce · Tile test uses alternating enabled pixel edges at 50 MHz. Fig 8/11.

pixel x · Coordinate crosses 24-pixel tile boundary, y fixed0; disabled clocks hold. Fig 11.

sub_x / tile_col · Tile subcounter wraps23□0 and column advances at the same edge as x. Fig 11.

vga_tile_addr · row×20+col. Boundary test tile0=floor, tile1=wall; no division hardware. Fig 11.

vga_tile_data · Tile RAM returns preceding column address. Independent window has floor prehistory. Fig 7.

maze_wall_q · Registered nonzero tile AND in_field_d. RAM then output FF gives two-clock coordinate-to-wall delay. Fig 11.

maze_rgb_q · Floor/wall stream colours. Floor222 is NOT selected as BG by literal Fig10. Fig 11.

bullet in_box · Bullet pixel bounding-box hit before FF; illustrative ROM shape=1 at these addresses. Fig 9.

bullet act · Source register alive until software clear effective at16. Delayed act must suppress ROM output. Fig 9.

in_box_d / shape · Bounding-box FF and synchronous shape ROM both delayed one clock. Shape stimulus chosen, not derived from absent ROM contents. Fig 9.

sprite act_d · Alive FF delayed one clock. act source cleared16 □ act_d low17. Fig 9.

bN_small_hit · ROM shape AND delayed box AND delayed activity, normal bullet mode. Fig 9.

bullet_valid (combined) · Additional combined output FF mentioned in Fig9 PartC; one clock after small_hit. Alignment with other sources is assumed.

HUD slot / hp · slot<hp gives full heart. Slot19 empty when hp19; geometry within HUD strip. Fig 12.

HUD in_hud · Inside HUD row and strip. Independent illustrative pixel inputs; not a consecutive raster coordinate sweep. Fig 12.

HUD lit_d · One-clock registered slot<hp. Initial full prehistory, then empty slot accepted23. Fig 12.

HUD shape (ROM) · Chosen synchronous heart ROM bit=1; last shape output remains for one clock after leaving region. Fig 12.

hud_valid_q · Registered in_hud_d AND shape, two clocks after geometry. Empty hearts remain valid. Fig 12.

hud_rgb_q · Final HUD colour FF consumes lit_d; independent of shape validity. Fig 12.

- ROM outputs use illustrative shape bits since block diagrams contain no ROM image data. Streams are independent diagnostic tests.

- RAM same-address cross-port write/read mode and RAM reset contents are unspecified; this test avoids that collision and assumes preloaded tiles.

- Unqualified X tile restart at h_end in Fig11 must be checked when pix_ce=0; boundary tests use a qualified restart convention.

I. CPU corner cases — loads, halfword lanes, x0, jump and exception detect

Seven independent four-clock tests; // separates transactions. Valid inputs established before window. Trap/recovery not specified by diagrams.

Signal · Meaning / source

WB transaction · Independent transactions through load/store/control blocks. P2/P5/P6.

stall_read_i · Chosen initial read wait only for load tests. Return qualified once next clock. P6 assumption.

wb_read_data · Test BRAM word; byte1=0x80, halfword1=0x8000. P6.

wb_write_en · Qualified writes; write x0 request may assert, but reads remain forced zero. Architectural x0 protection needs implementation check. P4.

wb_result · LB/LH sign extension versus LBU/LHU zero extension. x0 attempted data99 does not change x0 read result. P6/P4.

d_we / store_en · SH transaction pulse; no load peripheral write. P6.

d_addr · Aligned halfword at upper pair of word0x100. Misaligned access/trap behaviour unspecified. P6.

d_wdata · Halfword0xABCD replicated twice by store-data mux. P6.

d_wstrb · SH upper pair at addr[1]=1. Lower SH uses0011; SB uses0001/0010/0100/1000; SW1111. P6.

zero1 / zero2 · Both source addresses0 force both read operands0, overriding even matching WB forwarding. P4.

reg_rdata1 / 2 · Consumed x0 read-port values; x0 is architecturally zero. P4.

branch_taken · JALR redirect unconditional. No target instruction retirement shown in this isolated comparator test. P5.

fetch_pc_o (JALR) · rs1=201, immediate0 □ target (201+0)&~1=200. JALR clears low bit. P5.

reset_n · Exception test reset qualifier. CPU reset/hold/squash priority not determined by this isolated equation test. P2.

illegal_inst · Explicit decoder illegal input stimulus; suppressed by reset_n=0 at26. P2.

instruction_i == 0 · All-zero instruction also contributes to exception detect. A legal NOP0x13 does NOT. P2.

exception · (illegal_inst OR instruction_i==0) AND reset_n as drawn. Trap PC/cause/recovery outputs absent; no invented trap sequence. P2.

- Aligned data-memory tests only. MMIO is word-only; SB/SH MMIO behaviour should not be inferred from data-memory lane tests.

- Normal ALU add/shift/logic/compare are selected combinationally; numeric examples in sheet A cover ADD, MUL, SLT, OR, LUI, SRAI and XOR.